從把程式寫出來，到理解一套系統怎麼運作

A 國中｜第一次讓程式控制實體世界

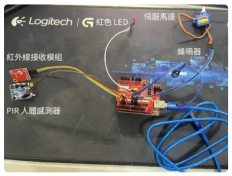


和兩位同學製作〈多功能智慧遮雨棚〉，我負責 Arduino 接線、程式與 App Inventor 手機控制。這是我第一次發現，程式不是只要自己能執行，還必須和感測器、接線、裝置與實體環境一起工作。

2023 IEYI 臺灣選拔賽　國中組便利生活類｜銀獎

▶ 延伸資料｜作品簡報 ↗　　▶ 延伸資料｜操作影片 ↗

B 高中｜防呆警報器：先拆開驗證，再逐步整合



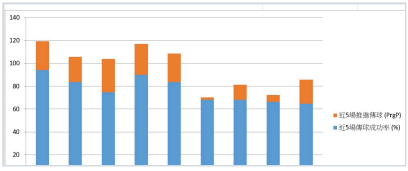
紅外線、PIR、蜂鳴器與伺服馬達

我原本一次把整套系統接起來，紅外線訊號卻一直讀不到，也不知道問題在哪一層。後來改成每個模組單獨驗證，再逐一加入人體感測器、蜂鳴器與伺服馬達，最後才整合，從此養成「先拆開、逐層驗證、再整合」的習慣。

▶ 延伸資料｜智慧節能防呆警報器 ↗　　▶ 延伸資料｜操作影片 ↗

C 高一｜因為足球，第一次碰到外部系統問題

我和朋友常爭論哪位球員最近狀態比較好。與其靠印象，不如把數據抓下來比：我自學 Python、pandas 與網頁爬蟲，從 FBref（足球數據網站）抓取球員近期與生涯數據，整理成可以比較的資料。



球員近五場各指標比較

單一球員成功 → 擴大查詢 IP 被限制 → VPN 仍失敗 → 理解請求頻率限制 → 加入請求間隔

那次我第一次具體理解：程式能跑，和整個系統能穩定運作，是兩件不同的事。

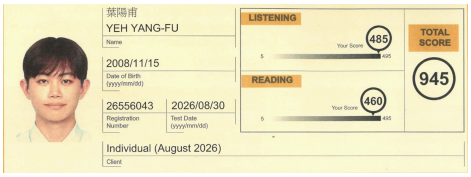
▶ 延伸資料｜GitHub 程式碼 ↗　　▶ 延伸資料｜專題報告書 ↗

D 英文｜自主學習的工具

TOEIC 945

GEPT 中高級
全民英檢・四項通過

做程式與 AI 專案後，我習慣直接閱讀 API 文件、技術說明、論壇與英文論文；英文成為我自主學習與查找問題的重要工具。



E 足球｜十年以上的長期投入

113 學年度
全國第六

114 學年度
全國第五

中等學校足球聯賽（五人制）・連續兩年全國決賽 8 強

幼兒園開始踢球，國三任花崗國中男子足球隊隊長，高中為正式校隊；做專題、科展與旺宏的同時，我仍持續校隊訓練與全國賽事。

▶ 延伸資料｜114 學年度全國決賽直播片段 ↗

從教學需求開始做自適應輔助學習系統

▶ 專案 GitHub ↗ ｜ **▶ 作品 Demo 頁面 ↗**

高二時我想要參加育秀盃，指導老師要我們先找身邊真正存在的問題。我問在高職教數學的父親，他說同一班學生程度差距很大，老師難以在同一堂課替不同學生安排不同進度的練習與回饋——這成為我開發「自適應輔助學習系統」的起點。

A 學生答完之後，下一步往哪走？

學生作答 → 留下作答紀錄 → 更新目前學習狀態 → 判斷下一步

已掌握觀念 → **下一題／下一技能**

觀念尚未掌握 找出弱點／前置概念 → **補強** → **AI 提示** → **推薦練習** → 再次作答

再次作答後，回到「**更新學習狀態**」，形成循環。

B Code-as-Content｜把「出題程式」當成題庫內容

題庫採用我自行設計的 Code-as-Content 做法：題庫先保存題型與教材內容，再由語言模型產生對應的 Python 出題程式；執行時帶入不同參數，即時產生新的題目與答案。

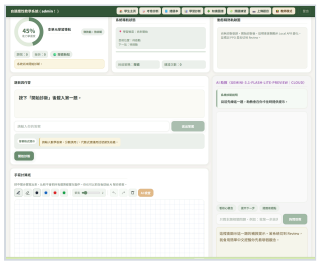
題型／教材資料 → **語言模型產生 Python 出題程式** → **帶入不同參數** → **即時產生題目＋答案**

同一支程式 → 調換題目參數＋對應答案 因此同一題型可以由同一支程式反覆產生不同題目與答案。

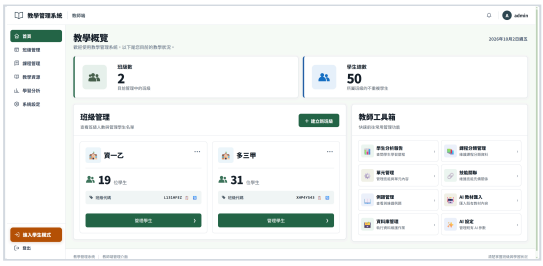
C 學生與教師實際能做什麼

學生端 依冊別、章節與技能進入練習，單選、填充、題組・圖片、動態圖形與作圖題・手寫／電子計算紙・AI 提示與補強・作答紀錄與後續練習

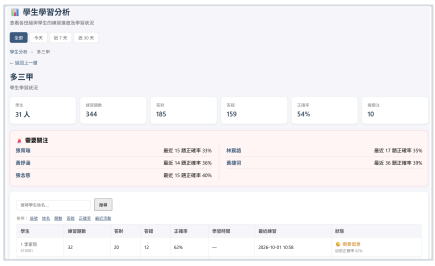
教師端 PDF／Word 教材匯入・建立班級與學生・教材章節與技能結構・教材內容審核・查看學生作答與學習紀錄・持續維護教材與題庫



學生端｜複習模式



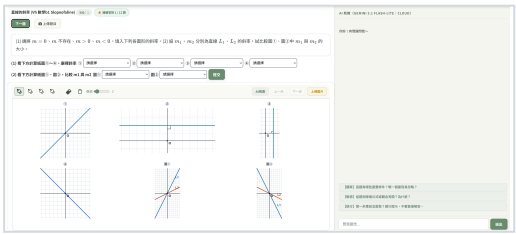
教師端｜班級管理與教師工具箱



學習分析｜需關注學生



教師端｜單元管理



學生端｜單元練習



教材｜章節與技能

從教材匯入到學習分析

教材匯入 → 題目與練習 → 學生作答 → 留下紀錄 → 教師分析

A	**教材結構化**	PDF／Word → 冊別 → 章節 → 小節 → 技能 → 例題／練習，保留來源；**B1–B4 已納入同一套體系。**
B	**練習與判分**	選擇・填充・題組・作圖・手寫計算紙・AI 助教；生成題須通過規格與判分檢查，處理數學等價（如 1/2 = 2/4）。
C	**教師端分析**	班級狀況・技能正確率・需關注學生・個別紀錄；修改教材、題型或判分後，重新測試相關流程。

2026 神通 AI 數位學院扶輪盃
高中組全國第一名

第 23 屆育秀盃創意獎
高中職 AI 應用類金獎

實際使用
50 **位學生**

D 版本演進｜同一套系統，問題一路改變

2025.10–12 **系統成形** 扶輪盃階段

問題　先讓整套系統真正可以被學生與教師操作。

改動　串起教材、題目、AI 協助、**學生端／教師端**與班級管理。

GitHub　本機專案整合 → 圖片上傳 → 提示詞功能 → 班級邀請碼

2026.01–04 **適性學習原型** 育秀盃階段

問題　學生答錯後，系統怎麼知道下一步該去哪裡？

改動　加入**能力狀態**、前置技能、複習與推薦、AI 錯誤診斷，並嘗試不同的適性學習架構。

GitHub　Code Generator v7.7 → v8.6 → 適性學習 → RAG／PPO／AKT

2026.05–09 **教材擴充與真實使用**

問題　教材與學生一多，資料、題型、判分與教師端開始變複雜。

改動　擴充 **B1–B4 教材**、技能結構、出題程式、數學判分、教師分析與手寫作答。

GitHub　B1 資料匯入 → 技能函數 → 自動出題 → 教師分析 → 手寫作答

2026.09–10 **真實上線與系統維護** 目前版本

問題　真正上線後，修改一個地方可能影響其他功能。

改動　強化出題程式規格、數學等價判定、**回歸測試**、題型覆蓋與品質檢查，並重構或移除不再適合的舊架構。

GitHub　題型覆蓋檢查 → 回歸測試 → 品質檢查 → 舊架構整理

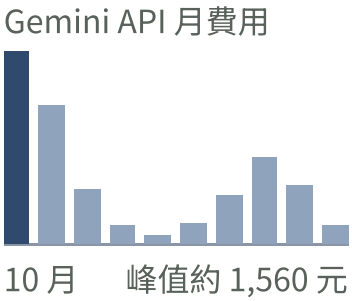
▶ 延伸資料｜扶輪盃作品說明書 ↗　▶ 延伸資料｜育秀盃官方作品頁 ↗　▶ 延伸資料｜系統實機展示影片 ↗

系統持續使用後，我開始遇到另一個問題：如果每次遇到新題型都重新呼叫大型模型產生 Python 程式，**API 成本、等待時間與生成錯誤都會持續累積。**

系統維運問題，成為研究起點

A **研究起點｜大型模型的成本與延遲**

學習系統的 Code-as-Content 每次生成程式，都要呼叫雲端大型模型；token、API 成本與等待時間隨使用持續累積。我想把部分工作移給本地小模型，實測後卻發現：**小模型的程式生成成功率並不穩定。**



B　研究問題

怎麼讓小模型少做一點，卻做得更可靠？

我觀察到，同一數學單元的程式結構與常見錯誤會反覆出現，於是把問題拆成兩個可以驗證的子問題：

Q1｜生成前
哪些重複工作可以事先抽離？

→ Scaffolding／Domain Functions
先整理好題型規則、工具、參數來源與部分流程，**降低模型自己規劃的負擔。**

Q2｜生成後
哪些局部錯誤可用固定規則修復？

→ AST Active Healer
只修固定規格能唯一判斷修法的局部錯誤，**把能確定修法的錯誤交給規則處理。**

Healer 不是自動修好所有錯誤；無法確定修法時就不介入。

C **初版驗證**

初版以三種模型，分為基礎提示組、鷹架提示組、鷹架提示 + Healer 組三組進行消融實驗，以自建評估指標來進行比較，以下為三個不同模型在自建指標的平均分數：

Bare	→	＋ Scaffolding	→	＋ Full Healer
41.00		94.39		97.41

MCRI（Math Code Reliability Index）為自建評估指標；97.41 是 MCRI 分數，不是通過率。

三種模型都出現相同的改善趨勢。

▶ 延伸資料｜東區科展作品說明書 ↗

當時，我以為這些結果已經足以支持方法有效。

97.41 是怎麼得到的？ 收到評審提問後，我重新驗證

校內科展
優勝
→
第 66 屆東區科展
優等
未晉級全國
→
第 25 屆旺宏科學獎
全國決賽 20 強
全領域 846 件
→
入圍之後
收到評審建議

A 評審提問中初版驗證的三個缺口，以及下一版怎麼改

① 測試資料獨立嗎？	→ 開發／測試分離，評估前凍結規則
② Healer 修好了什麼？	→ 逐筆追蹤每個原始失敗
③ 自建 MCRI 夠當證據嗎？	→ 聚焦 Healer，改用 PASS／FAIL

B 重新驗證：Math16 16 道會考題 × 3 模型 × 4 Prompt × 5 seeds = 960 單元

4 題｜規則開發
240 單元
→
凍結 Prompt 與 Healer 規則
→
12 題｜保留題正式測試
720 單元・Healer 不讀答案・同題不跨組

720	→	342	→	42	→	9	=	2.63%
正式測試		原始失敗		Healer 介入		失敗→通過		安全救援率

2.63% 與初版 MCRI 97.41 是不同指標，不能直接比較。

為什麼只有 2.63%？342 個原始失敗中，144 個（約 42%）是答案或演算法錯誤，需要重新理解題意或重新解題，本來就不在 Healer 的介入範圍。

能修｜輸出格式・局部結構・資料流・固定 API 規格可判定的錯誤

不修｜數學答案・核心演算法・題意理解・需重新推理的錯誤

C 新的發現｜主要差異來自模型能力與流程設計

補齊 Domain Functions 後，只給工具選單仍不夠；加上參數來源、呼叫順序、資料流與答案組裝的完整流程，部分中小型模型才進一步改善。

Domain Functions	完整流程 Scaffolding	Active Healer
提供領域工具	降低規劃負擔	只修可安全判定的錯誤

重新驗證後，我開始把注意力放在：一套 AI 軟體系統要怎麼分開處理模型能力、流程設計、程式修復與測試邊界；我也更確定自己想往軟體系統、程式分析與測試這些方向繼續學。

▶ 延伸資料｜旺宏決賽成果報告 ↗　▶ 延伸資料｜評審建議 ↗

系統做得越大，我越知道自己還缺什麼

A 三個需要靠大學補足的能力

① **演算法與資料結構**	我可以閱讀程式、拆問題，也能把多個模組整合成系統；但遇到需要把問題抽象成演算法、設計資料結構，並穩定完成實作時，仍不夠扎實。 **APCS｜觀念題 92 分／5 級・實作 135 分／2 級**　這個結果也很直接地反映這個差異：我在閱讀、理解與拆解問題上較成熟，演算法與正式資料結構訓練仍需要補強。
② **大型軟體系統與維護**	系統變大後，我開始碰到模組相依、資料流、版本修改、錯誤追蹤與測試的問題。很多做法是一路實作後摸索出來的，我想知道這些問題在正式的軟體工程與系統課程中，應該如何被更有系統地處理。 **對應**｜軟體工程・計算機組織・作業系統・軟體測試
③ **程式分析與 AI 系統可靠度**	Scaffolding 與 AST Active Healer 讓我開始碰到程式分析、模型能力邊界與測試設計，但目前很多理解仍是從專案與實驗一路摸索出來的。 **想正式理解**｜程式如何被分析・哪些錯誤可以用規則處理・模型與程式如何分工・測試如何界定可靠度

B 把「我做過的」和「我需要學的」對起來

高中已經碰過

- 大型網頁系統
- 教材／題庫／作答資料流
- AI 模型串接
- Code-as-Content
- 測試與驗證
- AST 程式修復
- 真實學生使用

→

大學需要正式補足

- 資料結構
- 演算法
- 離散數學
- 計算機組織
- 作業系統
- 軟體工程
- 程式分析
- AI 系統

C 為什麼是清大資工

高中專案把我帶到的問題，集中在**軟體系統、程式可靠度、測試、程式分析與 AI 系統**；我最想深入的，是讓這些環節一起可靠運作，單一 AI 模型只是其中一部分。這些問題都需要扎實的資工基礎才能真正處理好，而這些所學也能直接放回仍在使用中的系統。

高中專案讓我先遇到了很多資工問題；進入大學後，我想把這些靠實作摸索出來的經驗，用正式的資工方法重新理解一次。

下一步：把高中已經開始的問題繼續做下去

第一階段
把靠經驗做出的系統重新理解一次

現在的問題｜目前很多系統設計，是遇到問題後一路自學、實作、修改累積出來的。

要學｜資料結構・演算法・離散數學・計算機組織・作業系統

帶回專案｜重新整理教材、題庫與作答資料流・重新檢查模組邊界・改善資料結構與系統架構・建立更清楚的測試紀錄。**也重新檢查目前每個設計選擇背後的理由。**

第二階段
把「能跑」往「可靠」推進

現在的問題｜Code-as-Content、Scaffolding 與 Healer 已經遇到不同模型的能力差異、生成錯誤、修復邊界、評估設計與可重現性問題。

要學｜軟體工程・軟體測試・程式分析・AI 系統・可重現實驗方法

帶回專案｜建立固定實驗流程・擴大保留題測試・保留逐筆錯誤紀錄・分開模型錯誤、程式錯誤與流程錯誤・重新驗證 Scaffolding／Healer 的適用範圍

第三階段
回到真實學生

這套系統到底有沒有真的幫助學生？

高中階段已經做到｜教材匯入・章節／技能結構・Code-as-Content・多元作答・數學判分・學習紀錄・適性學習循環・AI 助教・教師端／學生端・真實學生使用

大學後要做深｜更完整的學生知識掌握模型・比較不同推薦策略・建立合理對照組・長期追蹤學習狀態・分開驗證「系統可運作」與「學生真的進步」

我想參加的比賽

大學後，我也想帶著這套系統繼續參加 InnoServe、經濟部智慧創新大賞，以及聯發科技「智在家鄉」。前兩項希望延續資訊系統與 AI 應用的方向；「智在家鄉」則希望把系統帶回花蓮，看看它能不能服務更多學生。

InnoServe	Best AI Awards	聯發科技「智在家鄉」
教育 AI／資訊應用	AI 技術與實際應用	花蓮在地教育應用

進入清大後，我希望繼續做這套高中已經開始的系統，把現在碰到的問題帶進更完整的資訊工程訓練裡，一個一個往下解。